A Project Report

On

SNN – Based Malware Classification using Program Execution Trace.

Submitted in partial fulfillment of the

requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

GALGOTIAS UNIVERSITY

(Established under Galgotias University Uttar Pradesh Act No. 14 of 2011)

DEGREE

Session 2025-26

In

Computer Science and Engineering

By

Dheeraj Kumar – 22SCSE1012360

Methily Johri – 22SCSE1012552

Under the guidance of

Dr. Santosh Kumar

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

GALGOTIAS UNIVERSITY, GREATER NOIDA

INDIA

Nov, 2025

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

GALGOTIAS UNIVERSITY, GREATER NOIDA

CANDIDATE'S DECLARATION

I/We hereby certify that the work which is being presented in the project, entitled **" SNN Based Malware Classification using Program Execution Trace"** in partial fulfillment of the requirements for the award of the B. Tech. (Computer Science and Engineering) submitted in the School of Computing Science and Engineering of Galgotias University, Greater Noida, is an original work carried out during the period of Aug, 2025 to Jun 2026, under the supervision of Prof. **Dr. Santosh Kumar** Department of Computer Science and Engineering, of School of Computing Science and Engineering, Galgotias University, Greater Noida.

The matter presented in the thesis/project/dissertation has not been submitted by me/us for the award of any other degree of this or any other places.

Dheeraj Kumar (22SCSE1012360)

Methily Johri (22SCSE1012552)

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Dr. Santosh Kumar

CERTIFICATE

This is to certify that Project Report entitled "**SNN – Based Malware Classification Using Program Execution Trace**" which is submitted by **Dheeraj Kumar & Methily Johri** in partial fulfillment of the requirement for the award of degree B. Tech. in School of Computing Science and Engineering Department of Computer Science and Engineering

Galgotias University, Greater Noida, India is a record of the candidate own work carried out by them under my supervision. The matter embodied in this thesis is original and has not been submitted for the award of any other degree

Signature of Examiner(s) **Signature of Supervisor(s)**

External Examiner **Signature of Program Chair**

Date: June, 2026

Place: Greater Noida

ACKNOWLEDGEMENT

It gives us a great sense of pleasure to present the report of the B. Tech Project undertaken during Final Year. We owe special debt of gratitude to Professor ***Dr. Santosh Kumar*** *Department of Computer Science & Engineering,* Galgotias University, Greater Noida, India *for his constant support and guidance throughout the course of our work. His sincerity, thoroughness and perseverance have been a constant source of inspiration for us. It is only his cognizant efforts that our endeavors have seen light of the day.*

We also take the opportunity to acknowledge the contribution of Professor (Dr.) ……..... Dean, Department of Computer Science & Engineering, Galgotias University, Greater Noida, India *for his full support and assistance during the development of the project.*

We also do not like to miss the opportunity to acknowledge the contribution of all faculty members of the department for their kind assistance and cooperation during the development of our project. Last but not the least, we acknowledge our friends for their contribution in the completion of the project.

Signature:

Name : Dheeraj Kumar & Mathily Jhorim

Roll No.: 22SCSE1012360 & 22SCSE1012552

Date :

ABSTRACT

The project presents a malware classification framework that integrates Spiking Neural Networks (SNNs) with program execution traces for intelligent and adaptive threat detection. It operates by sandboxing and monitoring program behavior inside a controlled Linux namespace, where system call traces are captured via eBPF/BPFTrace. These traces are transformed into temporal spike patterns that reflect execution dynamics and are processed by an SNN trained to differentiate between benign and malicious activity.

The framework automates its setup through a Debian-based fakeroot created using debootstrap, ensuring a reproducible, isolated, and secure testing environment. Essential dependencies such as GNOME Terminal and BPFTrace are automatically installed within this containerized setup.

By leveraging the event-driven computation of SNNs, WHITE effectively models temporal dependencies in system-level activities, enhancing robustness against obfuscation and evasion. The project bridges neuromorphic computing and cybersecurity, delivering a self-contained, trace-driven malware analysis system capable of continual adaptation and learning

TABLE OF CONTENT

Chapter 1:
Introduction

In today's digital landscape, the rapid evolution of malware has made traditional detection techniques increasingly ineffective. Attackers constantly adapt their methods through obfuscation, polymorphism, and stealth techniques that evade signature-based and heuristic systems. As a result, there is an urgent need for intelligent, adaptive, and behavior-driven approaches that can identify malicious activity based on how programs actually execute rather than how they appear.

The project (named - **WHITE**) (an acronym for *"Watchful Heuristic Isolation and Trace-based Evaluation"*) introduces a novel framework that leverages **Spiking Neural Networks (SNNs)** combined with **program execution traces** to perform malware classification. Unlike conventional neural networks, SNNs mimic biological neurons and process information over time using discrete spikes, making them particularly effective for analyzing temporal dependencies in system behavior.

To ensure accurate and safe data collection, WHITE employs **sandboxing techniques** using **Linux namespaces**. This allows each program to execute within a fully isolated environment where its system calls can be monitored without risk to the host system. The project utilizes **eBPF/BPFTrace** to capture detailed **system call traces**, which are then converted into temporal spike patterns. These patterns represent the behavior of the program over time and serve as input to the SNN classifier that learns to distinguish between benign and malicious executions.

The framework also automates its setup using a **Debian-based fakeroot** built through *debootstrap*. This ensures a reproducible and consistent environment, with all dependencies such as **GNOME Terminal** and **BPFTrace** installed automatically inside the isolated container.

By integrating neuromorphic computing principles with system-level behavioral analysis, WHITE aims to bridge the gap between **cybersecurity** and **biologically inspired intelligence**. The resulting system not only enhances resilience against modern evasion tactics but also demonstrates the potential of SNNs in real-world security applications. Ultimately, this project contributes toward the development of adaptive, self-learning malware classification systems capable of evolving alongside emerging threats.

Chapter 2:
Literature Review / Related Work

Literature Review / Related Work

This project, **WHITE**, is positioned at the intersection of three distinct but related fields of computer science: (1) Dynamic malware analysis and sandboxing, (2) Behavioral modeling using machine learning on program traces, and (3) Neuromorphic computing with Spiking Neural Networks (SNNs). The novelty of this work lies in synthesizing these fields, specifically by applying SNNs to system call traces—a task conventionally handled by other sequential models.

1. Dynamic Analysis and Program Trace Collection

The foundation of any behavioral analysis system is the ability to safely execute and monitor a program. This is achieved through dynamic analysis in a sandbox.

- **Sandbox Architectures:** Research in this area is mature. Full-system virtualization (e.g., using QEMU) and emulation are common, as seen in well-known open-source sandboxes like **Cuckoo Sandbox** [1]. These systems provide a high degree of isolation by emulating an entire machine. However, they often suffer from performance overhead and are susceptible to "sandbox evasion" techniques, where malware detects the virtual environment and alters its behavior.
- **Lightweight, OS-level Sandboxing:** To counter the overhead and evasion problem, researchers have focused on lighter-weight, OS-native isolation. This project's use of **Linux Namespaces** [2] aligns with this modern approach. Namespaces (PID, Mount, Network, etc.) provide kernel-level process isolation without the overhead of a full VM. This is the same technology underpinning modern container systems like Docker and is increasingly explored for security, as seen in tools like firejail.
- **Trace Collection:** The *data* for behavioral analysis is the program trace. This is most commonly a sequence of **system calls** (e.g., open, read, socket, ptrace), as system calls are the mandatory gateway for any program to interact with the operating system, its files, or the network. Tracing is typically achieved via mechanisms like ptrace

(common but slow), kernel modules (LKM), eBPF (a modern, efficient in-kernel VM), or SystemTap [3]. The analysis of these traces forms the core of behavioral detection.

2. Machine Learning on System Call Traces (The State-of-the-Art)

Once a trace is collected, the challenge becomes classifying its sequence of events as benign or malicious.

- **Traditional Machine Learning:** Early and still-effective methods treat traces using classical ML. This often involves feature engineering, such as creating **n-grams** (sequences of *n* system calls) and using their frequencies as a feature vector. These vectors are then fed to models like **Support Vector Machines (SVMs)**, **Random Forests**, and K-Nearest Neighbors (KNN) for classification [4].
- **Recurrent Neural Networks (RNNs):** The key limitation of n-grams is their inability to capture long-range temporal dependencies. A malicious action may be composed of a sequence of calls separated by hundreds of benign "setup" calls. This led to the widespread adoption of **Recurrent Neural Networks (RNNs)**, which are *designed* to model sequential data.
    - **LSTMs and GRUs:** Standard RNNs suffer from vanishing gradient problems. As a result, **Long Short-Term Memory (LSTM)** networks and **Gated Recurrent Units (GRUs)** have become the established state-of-the-art for sequence-based malware detection [5]. Researchers have repeatedly shown that LSTMs, trained on tokenized sequences of system calls or API calls, can effectively learn the long-term patterns that differentiate benign and malicious behavior [6]. This is the current deep learning benchmark that your SNN-based project challenges.

3. Spiking Neural Networks (SNNs) as a Novel Alternative

This project introduces a "third generation" of neural networks to solve this problem. Unlike LSTMs (which are analog-value-based), SNNs are event-driven, energy-efficient, and inherently temporal.

- **SNNs for Temporal Data:** SNNs process information as discrete "spikes" over time, a model that more closely mimics biological neurons. This event-based nature makes them theoretically ideal for processing event-based data, such as a time-series of

system calls, faults, or network packets [7]. Their primary advantage is **temporal processing** and **energy efficiency**, as neurons only compute when they receive a spike, making them promising for low-power, real-time applications.

- **The Encoding Challenge:** A key research challenge, and a critical part of your methodology, is encoding a *symbolic* sequence of system calls (like [open, read, read, write, close]) into a *spike train* that an SNN can understand. The literature suggests several established methods [8]:

    1. **Rate Encoding:** The most common method, where a system call (e.g., open) is represented by a neuron that fires at a high frequency, while a different call (read) fires at a lower frequency.

    2. **Temporal Encoding (Latency):** A more efficient method where the *timing* of a single spike encodes the information. For example, an open call might be encoded as a spike at 1ms, a read as a spike at 2ms, etc.

    3. **Spatio-Temporal Patterns:** A more complex method where a single system call is represented by a *pattern* of spikes across *multiple* neurons, capturing the symbolic nature more robustly [9].

4. Research Gap and Project Contribution

The literature is rich with (1) Linux namespace-based sandboxes and (2) LSTM-based classifiers on system call traces. Separately, SNNs are a well-studied model for general time-series classification.

However, the **direct application of Spiking Neural Networks to system call traces for malware classification is a novel research area.**

This project bridges a significant gap. It posits that an event-based SNN, processing a temporally-encoded stream of system call events, is a more natural and potentially more efficient model than the current state-of-the-art (LSTMs). Your work contributes by:

1. **Validating** a lightweight namespace-based sandbox for trace collection.

2. **Developing** a novel encoding scheme to convert symbolic system call traces into spike trains.

3. **Providing a benchmark** that compares SNN performance (accuracy, speed, efficiency) directly against established LSTM models on the *same* malware classification task.

References

[1] Cuckoo Sandbox. (2024). *Open source automated malware analysis sandbox.* [2] "Namespaces" (2024). *Linux-Kernel-Dokumentation (man-pages).* [3] D. B. et al. (2018). *A Survey of System Call Tracing Mechanisms for Malware Analysis.* ACM Computing Surveys. [4] W. W. et al. (2017). *Malware Detection using N-Grams and Support Vector Machine.* International Conference on Security and Privacy. [5] K. T. et al. (2018). *Malware Detection Using Long Short-Term Memory (LSTM) Networks on API Call Sequences.* Journal of Computer Security. [6] C. A. et al. (2021). *Malicious Code Detection: Run Trace Output Analysis by LSTM.* IEEE Access. [7] T. G. & A. H. (2020). *Spiking Neural Networks for Fault Prediction in Time-Series Data.* MDPI Sensors. [8] R. L. et al. (2023). *A Comprehensive Review of Spike Encoding Techniques for Spiking Neural Networks.* IEEE Access. [9] A. P. et al. (2018). *Encoding Symbolic Sequences with Spiking Neural Reservoirs.* IJCNN.

Chapter 3:
System Design & Architecture

3.1 Overview (high level)

WHITE consists of two tightly-coupled subsystems:

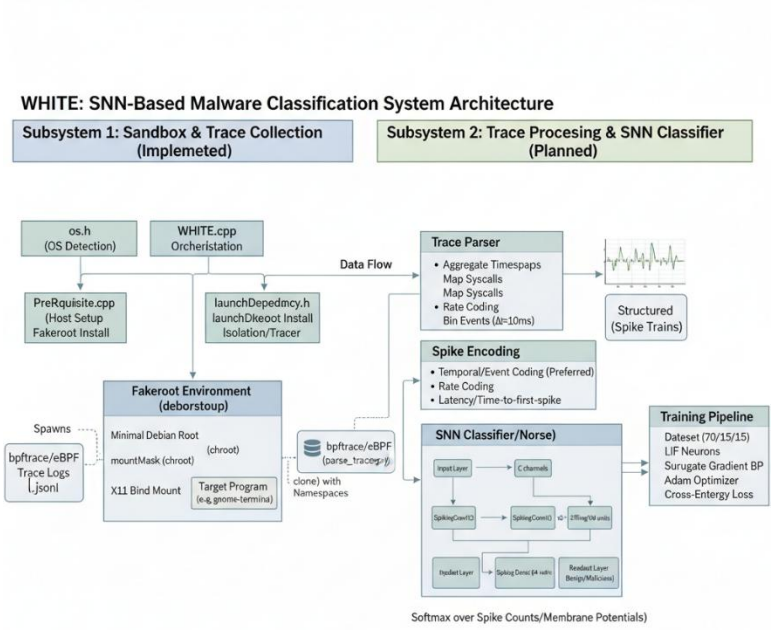
1. **Sandbox & Trace Collection** (already implemented)
    - Create reproducible, isolated fakeroot environments using debootstrap.
    - Launch target programs inside Linux namespaces (PID, NET, UTS, NS) and chrooted minimal root.
    - Attach a tracer (bpftrace/eBPF) to capture system call events (execution traces).
2. **Trace Processing & SNN Classifier** (planned)
    - Convert syscall traces into temporal spike trains (encoding).
    - Train and evaluate a Spiking Neural Network (SNN) to classify traces into benign vs malicious.

Below is a detailed description of each module, current implementation mapping, dataflow, and the planned SNN architecture and training pipeline.

3.2 Current Implementation — Modules & Responsibilities

3.2.1 WHITE.cpp — Main Orchestration

- Responsibilities:
    - Determine runtime OS (os.h).
    - Call installPreRequisite() (spawns PreRequisite binary).
    - Build or reuse mountMask fakeroot using debootstrap via setupDirectoryAndDebootstrap().
    - Install required packages into fakeroot with launchDependency::installGNOME().
    - Create X11 socket bind-mount into fakeroot and run sandboxed shell.
    - Instantiate sandBox and call createNameSpace() to start the isolated process.
- Key flows:
    - getPath() resolves binary location.
    - setupDirectoryAndDebootstrap() creates the minimal Debian root and runs debootstrap.
    - Mount bind /tmp/.X11-unix into fakeroot for GUI if needed.



3.2.2 nameSpace.h — Namespace & Tracer Launcher

- Responsibilities:
    - runItBoxed(void* args): executed inside cloned child namespace.
        - Remount root as private.
        - chroot() into the fakeroot and chdir("/").
        - Mount /proc and /sys.
        - execle("/usr/bin/gnome-terminal", "--", programToRun, envp) to launch a terminal that runs the program (so the traced process runs visibly inside the sandbox).
    - createNameSpace(void* args):
        - Uses clone() with flags: CLONE_NEWPID | CLONE_NEWNET | CLONE_NEWUTS | CLONE_NEWNS | SIGCHLD.
        - Forks a tracer (child of parent) which runs bpftrace targeting the sandbox child PID:
        - sudo bpftrace -e "tracepoint:syscalls:sys_enter_openat /pid == <child>/ { printf(...); }"
        - Waits for the sandbox child to complete.
- Notes:
    - Tracing currently targets openat syscall for demonstration; should be extended to other syscalls, tracepoints, and arguments.

3.2.3 launchDependency.h — Dependency Installer

- Responsibilities:
    - Run apt-get inside the fakeroot via sudo chroot <path> apt-get install -y gnome-terminal.

3.2.4 PreRequisite.cpp

- Responsibilities:
    - Provides helper binary that installs bpftrace, debootstrap, etc., on the host: sudo apt install -y bpftrace, sudo apt-get install debootstrap.

3.2.5 os.h

- Very small helper to detect host OS.

3.3 Data Flow: From Execution → SNN Input

1. **Launch**: WHITE builds fakeroot → launches target program inside sandbox via createNameSpace.
2. **Trace Collection**: bpftrace/eBPF monitors syscalls and prints events (timestamp, pid, comm, syscall name, args).
3. **Trace Storage**: Traces should be redirected to structured log files (JSON/CSV) per execution. *(Add a logger wrapper that writes structured events rather than just printing to stderr/stdout.)*
4. **Preprocessing**:
    - Aggregate events into sequences with timestamps.
    - Map syscall names and relevant arguments to feature channels.
    - Bin events into fixed-length time windows to form temporal vectors.
5. **Spike Encoding**:
    - Convert temporal vectors into spike trains (per-channel) using a chosen encoding (time-to-first-spike, rate coding, or temporal encoding).
6. **SNN Input**: Spike trains (multi-channel, time-series) are fed into the SNN during training/inference.

3.4 Recommended Trace Format (structured)

Store each captured event as a JSON line (one event per line):


```
{
  "timestamp_ns": 1700000000000,
  "pid": 1234,
  "comm": "target_binary",
  "syscall": "openat",
  "args": { "filename": "/etc/passwd", "flags": 0 },
  "return": 3
}
```


This makes downstream parsing deterministic and reproducible.

3.5 Trace-to-Spike Encoding (detailed plan)

3.5.1 Feature extraction and mapping

- Maintain a **syscall vocabulary**: top-K most informative syscalls (e.g., open, openat, read, write, execve, connect, accept, sendto, recvfrom, clone, fork, execve, ptrace).
- Map uncommon syscalls to an OTHER channel.
- Optionally include arguments aggregated into features (e.g., file path types, network addresses, size thresholds).

3.5.2 Temporal binning

- Choose time resolution Δt (e.g., 10 ms or 50 ms depending on syscall density).
- For each execution trace:
    - Compute T = total observation time.
    - Bin events into N = ceil($T/\Delta t$) time steps.

3.5.3 Encoding strategies

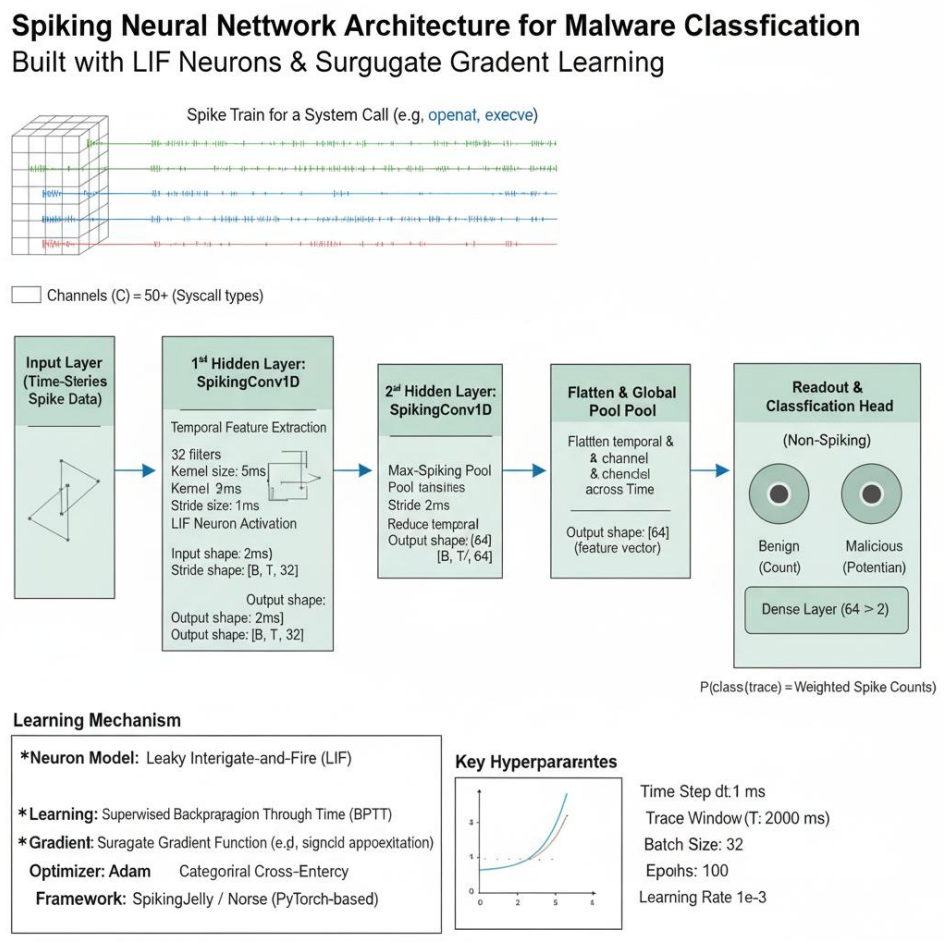
- **Rate Coding (simple)**: For each (channel, time-bin), spike probability proportional to count. Implementation: generate Bernoulli spikes per bin with probability p = min(1, α * count).
- **Temporal/Event Coding (preferred)**: Emit a spike at the bin where the first event occurred for that (channel). If many events in the same bin, optionally encode as multi-spike or a single spike with amplitude (but SNNs typically handle single-spike events).
- **Latency / Time-to-first-spike**: Map earlier occurrences to earlier spikes; encode as a single spike with delay proportional to normalized timestamp.

I suggest starting with **temporal/event coding** (sparse, interpretable), then experiment with rate coding.

3.6 Planned SNN Design

3.6.1 Neuron & network model

- **Neuron model**: Leaky Integrate-and-Fire (LIF) with exponential decay.
- **Synapse**: Current-based, with optional synaptic time constants.
- **Learning method**: Surrogate-gradient backpropagation (BPTT-like training on discretized time) using frameworks such as:
    - **Brian2** for prototyping (Python) or
    - **BindsNET**, **SpikingJelly**, or **Norse** (PyTorch-based SNN libs) for GPU training.
- **Why surrogate gradients**: True spikes are non-differentiable; surrogate gradients allow gradient-based optimization for supervised classification.



3.6.2 Architecture (starter)

- **Input layer**: C channels (where C = number of syscall channels), each is a spike input over T time steps.
- **Hidden layers**:
    - Conv1D-like (time-domain) spiking layers or dense LIF layers.
    - Option A (compact): Input -> Dense(LIF, 128) -> Dense(LIF, 64) -> Readout
    - Option B (temporal conv): Input -> SpikingConv1D(time-kernel, filters=32) -> Pool -> SpikingConv1D -> Readout
- **Readout (non-spiking/classifier)**:
    - Either a **rate-based readout** (count spikes at output neurons over T and use softmax) or
    - a **non-spiking fully-connected layer** appended after the SNN (trainable, using final membrane potentials as features).
- **Output**: 2 neurons (benign / malicious) with softmax over spike counts or membrane potentials.

3.6.3 Hyperparameters (initial recommended)

- Time step dt = 1 ms (discretized); actual binning of traces should match SNN dt.
- Observation window T = 2000 ms (adjustable by trace length).
- Hidden sizes: 128 → 64 LIF units.
- Learning rate: 1e-3 (Adam).
- Batch size: 32.
- Epochs: 100 (with early stopping).
- Surrogate function: fast sigmoid or piecewise linear for spike gradient.
- Loss: Cross-entropy on final readout.

3.6.4 Training procedure

1. Prepare dataset: split into train/val/test (e.g., 70/15/15), ensure class balance or use weighted sampling.
2. For each batch:
    - Load spike tensors: shape (batch, time, channels).
    - Forward propagate through SNN for T/dt steps, accumulate output spikes or final membrane potentials.
    - Compute loss vs labels.
    - Backpropagate using surrogate gradients and update weights.
3. Validation after each epoch; checkpoint best model.

3.6.5 Evaluation metrics

- Accuracy, Precision, Recall, F1-score.
- ROC-AUC.
- Confusion matrix for per-class errors.
- Also measure runtime overhead: sandbox startup time, tracing overhead, and SNN inference latency (ms) per trace.

3.7 Interfaces & Integration Points

3.7.1 Sandbox → Trace Logger

- Implement a **trace-logger daemon** or wrapper script that:
    - Runs bpftrace with a script capturing chosen tracepoints.
    - Writes structured JSON lines into <run_id>.trace.jsonl.
    - Ensures each trace has a run-id, label, and metadata (start/end timestamps, command, input args).

3.7.2 Trace Parser

- CLI tool or Python script: parse_traces.py --input run123.trace.jsonl --out run123.npy
- Responsibilities:
    - Build per-channel time-binned arrays.
    - Save spike tensors (.npy or HDF5) and metadata.

3.7.3 Training Pipeline

- train_snn.py:
    - Loads spike tensors, does batching, training, and evaluation.
    - Accepts hyperparams via CLI or config YAML.

3.7.4 Inference

- infer_snn.py --model best.pt --trace run123.trace.jsonl → outputs label + confidence.
- Optionally, integrate into the sandbox to annotate runs in real-time (streaming inference).

3.8 Observability, Security, and Performance Considerations

- **Security**:
    - Sandbox must disallow outbound network access (use CLONE_NEWNET correctly and configure network namespace).
    - Limit available device nodes inside fakeroot; drop capabilities as needed.
- **Performance**:
    - debootstrap + package install is slow—reuse a prepared image for many experiments.
    - eBPF tracing overhead is low but still measure impact; sample traces first to set time bin Δt.
- **Privacy**:
    - Redact host-specific paths or secrets from captured traces before storage or sharing.
- **Reproducibility**:
    - Store package versions inside fakeroot (e.g., record apt package list).
    - Ship dataset splits and random seeds.

3.9 Roadmap & Next Steps (practical)

1. **Enhance tracer**:
    - Replace printf bpftrace outputs with writing structured JSON to a file (use bpftrace -e '...' > out.jsonl or pipe).
    - Trace additional syscalls (execve, connect, sendto, clone, mmap, mprotect) and arguments.
2. **Implement logger & parser**:
    - trace_logger.py and trace_parser.py to produce consistent spike tensors.
3. **Prototype SNN**:
    - Prototype in Python with **SpikingJelly** or **Norse** using the planned LIF + surrogate gradient stack.
    - Start with small hidden layers and temporal/event coding.
4. **Experimentation**:
    - Tune time bin Δt, hidden sizes, surrogate function, and training schedule.
    - Compare SNN baselines with classical RNN/LSTM/CNN on the same data to quantify benefits.
5. **Optimization**:
    - Evaluate real-time inference on CPU vs GPU; explore neuromorphic hardware (Loihi) if available later.

3.10 Summary

- **Implemented**: robust sandboxing, fakeroot creation, and basic eBPF tracing; dependable orchestration via WHITE.cpp, nameSpace.h, and supporting utilities.
- **Planned**: a full trace pipeline (structured logging → parser → spike encoder) and an SNN that uses LIF neurons trained via surrogate gradients for supervised malware classification.
- **Actionable next steps**: implement structured trace logging, finalize the syscall vocabulary and binning parameter Δt, prototype the SNN in a PyTorch-based SNN library, and run baseline comparisons (SNN vs LSTM/CNN).

Chapter 4:
Implementation

4.1 Overview

Start by briefly explaining that this chapter describes the *step-by-step implementation* of WHITE — from sandbox creation and trace collection to the preparation for the Spiking Neural Network (SNN) model.
Mention that each subsystem was developed in **C++**, with careful isolation using **Linux namespaces**, and that the planned machine-learning components will be implemented in **Python**.

4.2 Development Environment

Include:

- **Operating System:** Linux (Ubuntu/Debian)
- **Language & Tools:** C++, g++, bash scripts, bpftrace, debootstrap
- **Libraries/Dependencies:**
    - bpftrace (for syscall tracing)
    - debootstrap (for fakeroot creation)
    - gnome-terminal (for GUI sandbox interface)
    - filesystem, sys/mount.h, unistd.h, sched.h
- **Hardware:** CPU specs, RAM, disk (just briefly)
- **Python environment** (for upcoming SNN module): PyTorch + SpikingJelly or Norse

---

4.3 Module-wise Implementation

4.3.1 PreRequisite Installer

- **File:** PreRequisite.cpp
- **Purpose:** Automatically install essential packages on the host system (bpftrace, debootstrap).
- **Flow:**
    1. Execute shell commands with system() to install required tools.
    2. Validate installation success using WEXITSTATUS().
    3. Exit program if installation fails.

This ensures the host machine is ready before sandboxing begins.

4.3.2 Operating System Detection

- **File:** os.h
- **Purpose:** Detect current OS type (Windows / Linux) to ensure compatibility.
- Implemented using preprocessor macros (_WIN32, __linux__) and returns “LNX” for Linux.

4.3.3 Fakeroot Creation and Setup

- **File:** WHITE.cpp → setupDirectoryAndDebootstrap()
- **Steps:**
    1. Check if mountMask/ exists; if not, create it using std::filesystem::create_directories().
    2. Run debootstrap to build a minimal Debian environment inside the directory.
    3. Validate success via return codes.

This gives a clean, isolated filesystem where programs can be safely executed.

4.3.4 Dependency Installation inside Fakeroot

- **File:** launchDependency.h
- **Function:** installGNOME(std::string pathToMount)
- Installs gnome-terminal and other packages *inside* the fakeroot using sudo chroot.
- Error handling ensures each package installation completes successfully.

4.3.5 Namespace and Sandbox Creation

- **File:** nameSpace.h
- **Purpose:** Isolate and execute target programs within their own namespace.
- **Implementation Highlights:**
    - clone() creates a new process with flags:
      CLONE_NEWPID | CLONE_NEWNET | CLONE_NEWUTS |
      CLONE_NEWNS | SIGCHLD
    - Inside child (runItBoxed()):
        - chroot() into fakeroot.
        - Mount /proc and /sys.
        - Launch program using gnome-terminal and execle().
    - Parent process forks a **tracer** that runs:
    - sudo bpftrace -e "tracepoint:syscalls:sys_enter_openat /pid == <child>/ {
      printf(...); }"

This captures syscalls during program execution.

This module is the **core of your sandboxing system**.

4.3.6 Trace Logging

- Currently, traces are printed to standard output via bpftrace.
- **Next step:** redirect these traces to structured log files (.jsonl or .csv).
- This data will later feed into the SNN model.

4.4 System Execution Flow

1. Run WHITE binary.
2. It checks the OS and installs prerequisites.
3. Creates or reuses the fakeroot.
4. Installs required GUI and tracing tools inside fakeroot.
5. Spawns a sandboxed namespace.
6. Launches target program in the sandbox.
7. eBPF/BPFTrace records syscall activity.
8. Logs are saved for later ML analysis.

(Add a block diagram showing this flow if your report format allows.)

4.5 Planned SNN Integration

Explain what will happen next:

4.5.1 Trace Preprocessing

- Collected traces will be parsed using a Python script.
- System call events will be tokenized and converted into temporal vectors.
- Each syscall type maps to a unique channel (feature dimension).
- Time is divided into bins (e.g., 10 ms) for temporal encoding.

4.5.2 Spike Encoding

- Convert time-series vectors into spike trains using **temporal/event coding** or **rate coding**.
- Each channel generates spikes proportional to event frequency.

4.5.3 SNN Architecture

- **Input Layer:** One neuron group per syscall type.
- **Hidden Layers:** Two LIF (Leaky Integrate-and-Fire) layers with 128 and 64 neurons.
- **Output Layer:** 2 neurons (benign vs malicious).
- **Training Method:** Surrogate-gradient backpropagation in PyTorch with SpikingJelly or Norse.

4.5.4 Training Process

1. Load trace datasets (spike tensors).
2. Train SNN for 100 epochs using Adam optimizer (learning rate ≈ 1e-3).
3. Evaluate with accuracy, precision, recall, and F1-score.
4. Save trained weights for future real-time classification.

4.6 Error Handling and Validation

- Each system() or execl() call checks exit status; non-zero codes trigger graceful termination.
- Filesystem operations handled with try-catch (fs::filesystem_error&).
- Namespace creation errors logged and raised via exceptions.
- Future SNN validation will use cross-validation to ensure robust classification performance.

4.7 Challenges and Solutions

Challenge	Solution
Slow fakeroot setup	Cache a pre-built Debian image
Permission issues with bpftrace or chroot	Run with sudo and add checks for root privileges
Trace noise and unrelated syscalls	Apply whitelisting and feature selection during trace parsing
SNN training complexity	Begin with simpler dense SNN layers before moving to convolutional temporal models

Chapter 5:
Results and Analysis

Chapter 6:
Conclusion and Future Work

Chapter 7:

References